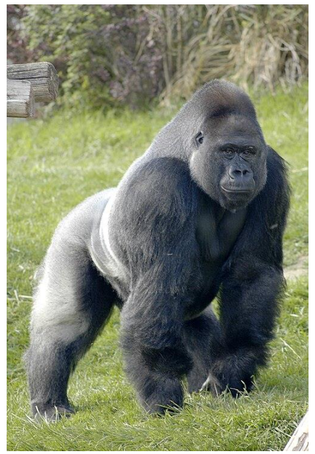
Mordred: NoC SST Element

Tim Dysart and David Donofrio
Tactical Computing Labs

SST User Group Meeting – Oct. 2025

Another Networking Element?

- **Shogun**
  - Crossbar
- **Kingsley**
  - 2D Mesh
  - Unclocked routers
  - No VCs
  - Limited active development? Future plans?
- **Merlin**



- Primary target is system interconnect
- NoC capabilities exist, but not the primary goal
- Unclocked routers; VC count dictated by topology
- Can be challenging to use

https://commons.wikimedia.org/wiki/File:Gorille_des_plaines_de_l%27ouest_%C3%A0_l%27Espace_Zoologique.jpg

Enter Mordred

Purpose

- Full featured element for on-{chip,package} networks
  - An SST based version of Booksim2 (https://github.com/booksim/booksim2)
- Replace Kingsley and Shogun with a more "universal" component
- Reduce the scope of Merlin – allow it to focus on system interconnect

Functional Goals

- Adhere to the SimpleNetwork interface
- Clocked router
- Multiple topologies
- Configurable – VN count, VC count (per VN), buffer depths, arbitration policies, latency
- Routing and Switching options – priority based, packet vs flit
- Credit based flow control
- Interface to all "chip" components: compute cores, memories, NICs, etc

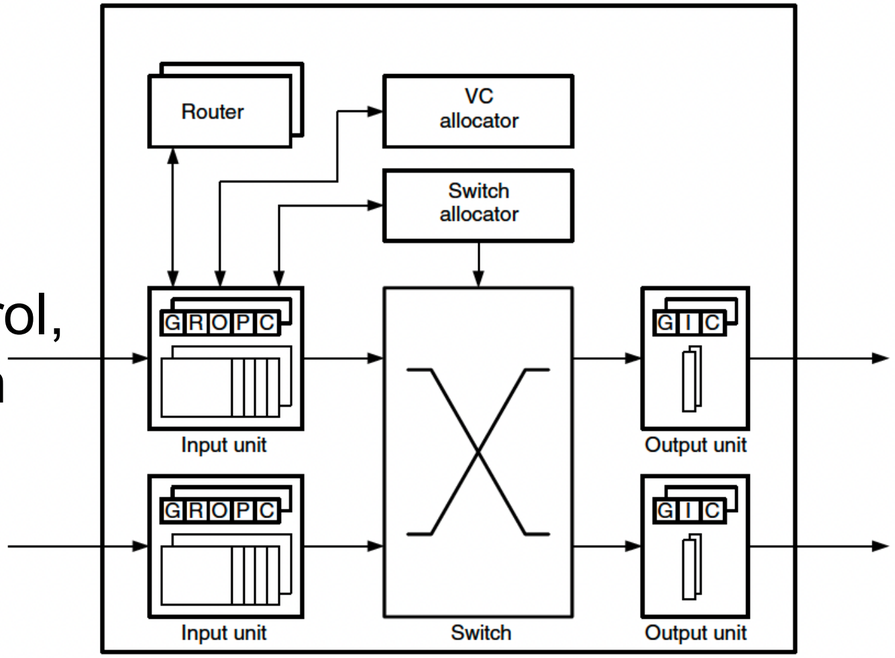
Mordred (Sub)Components

simple_rtr

- Router: probably needs a new name
- Subcomponent slots for Topology, Port Control, VC Allocation, Crossbar Allocation/Arbitration

Subcomponents

- Topologies: 2D Mesh, 2D Torus, Flattened Butterfly
- VC Allocator: Round Robin
- Crossbar Allocator/Arbitration: Round Robin
- Port control (input and output units)
- MordredNIC: endpoint interface



Principles and Practices of Interconnection Networks, Dally and Towles, 2004

Internal Behavior

- **Basic Overview**
  - Independent VN,VC "sets"
  - Assumes duplicated logic for routing
  - Per cycle try to advance the lead flit in each set
  - Packet based switching (once a set has won the switch, the port pair is held until the packet is fully transferred through it)

- **Key Data Structure: RtrOwnedSharedObjs**
  - 2D-Vector for flits requesting output VC
  - 2D-Vector for flits requesting the crossbar switch
  - Enabler for swapping out VC allocators, switch allocators
  - Merlin's hr_router has similar data structures

Experiments and Results

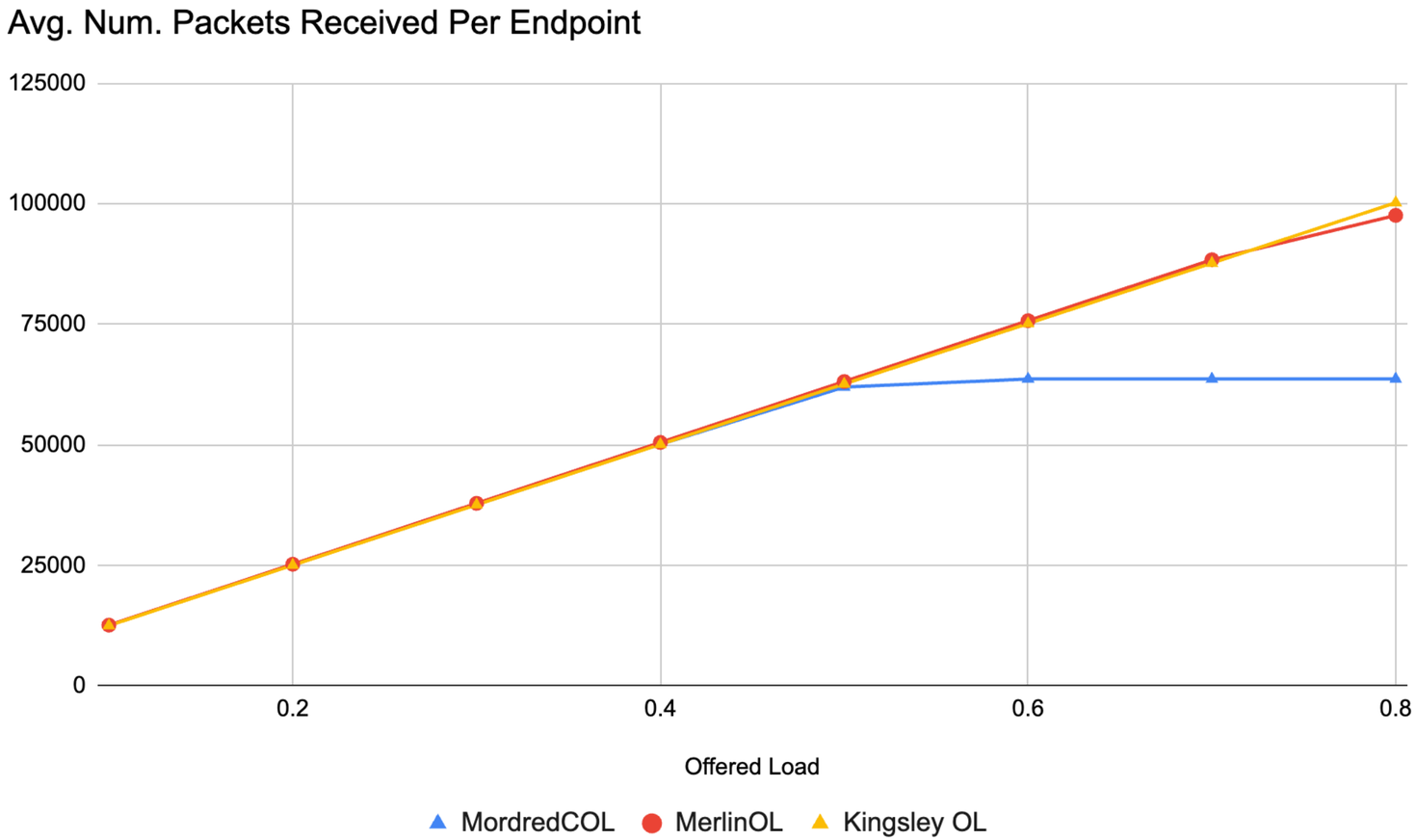
- Created a new Merlin component *clocked_offered_load* (COL in following charts) based on *offered_load (OL)*
  - Add a clock parameter and send packets/flits at the clock rate desired rate
  - Developed as a comparison point to offered_load with an equivalent link bandwidth
    - COL results with 1GHz clock == OL results with a link bandwidth of 2GB/s (16 bit channel)

- Compare Merlin, Mordred, and Kingsley

- 3x3 Mesh Topology
- Router Buffers:
  - 16 flit input buffer (per set)
  - 1 flit output buffer
- Endpoint NIC buffers of 1KiB
- Fixed packet length of 4 flits
- Uniform traffic generator
- 1GHz clock
- 800ps link latency (less than a single clock)

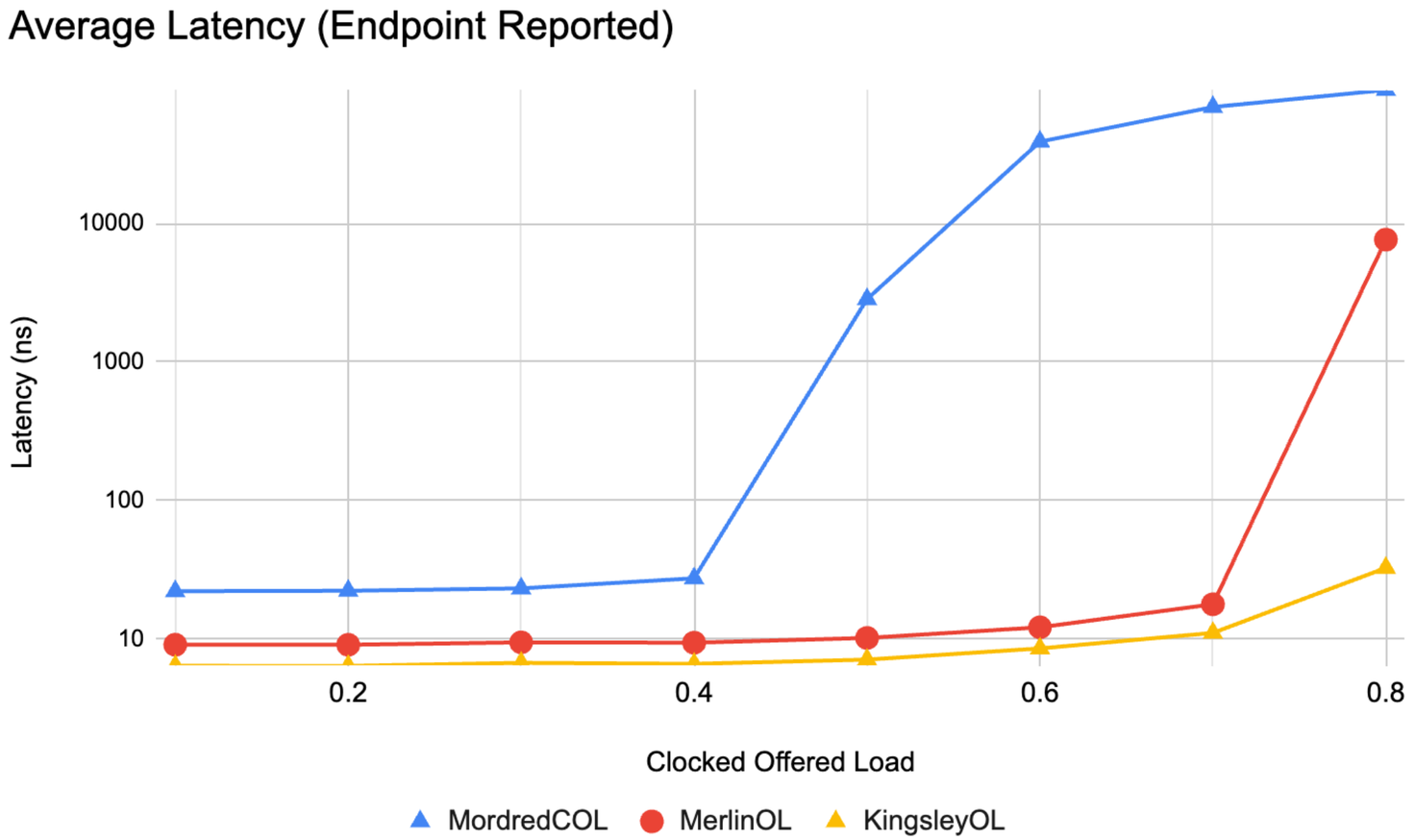
- 1us warmup, 500us run, 50us drain time

Packets Received Per Endpoint



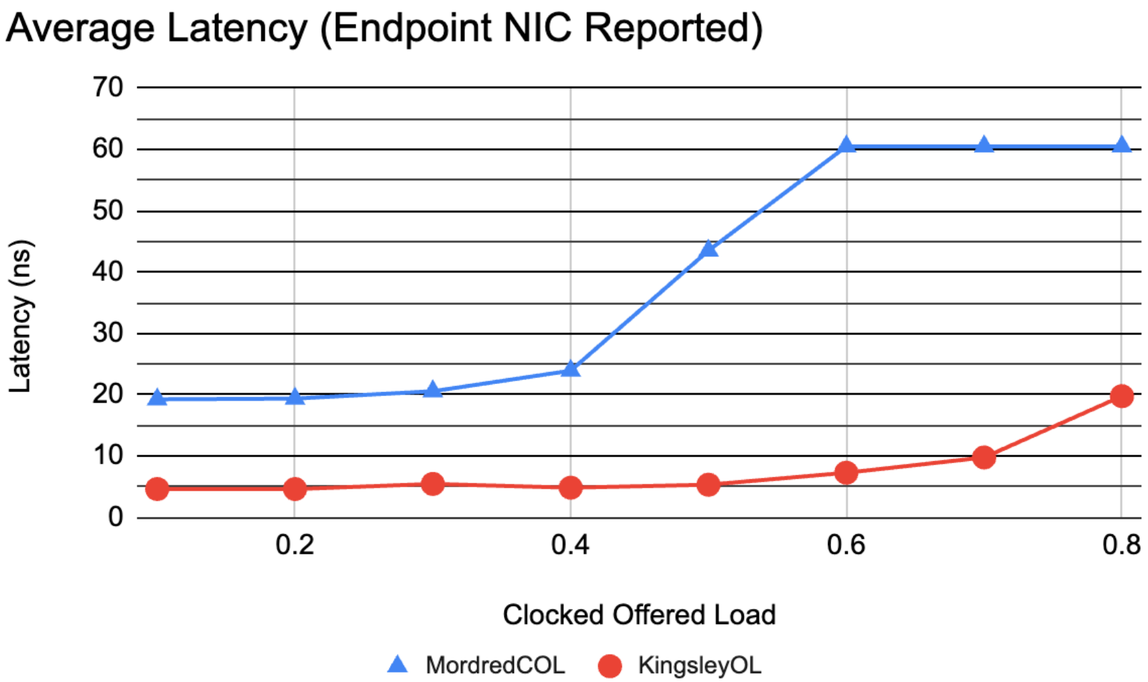
- Network starts to saturate around a load of 0.5 for Mordred; hence the flattening out
- Merlin and Kingsley do not noticeably saturate

Avg. Latency – Endpoint Reported



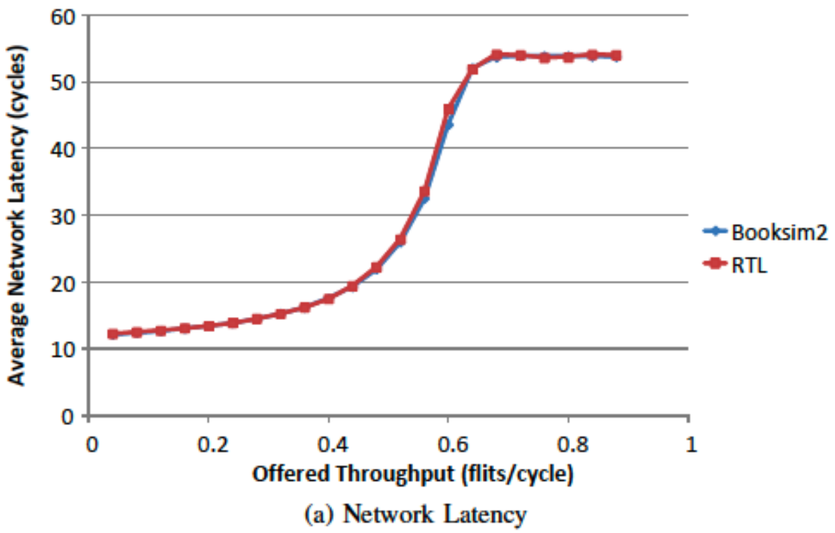
- Latency reported by the endpoints
- Likely includes delay while buffered in the NIC

Avg. Latency – Endpt NIC Reported



- Mordred timing starts when HEAD flit is sent out of the NIC and ends when TAIL flit is received at the NIC
- Kingsley does not saturate in an expected region; average latency is lower than expected

Booksim2 Results



(a) Network Latency

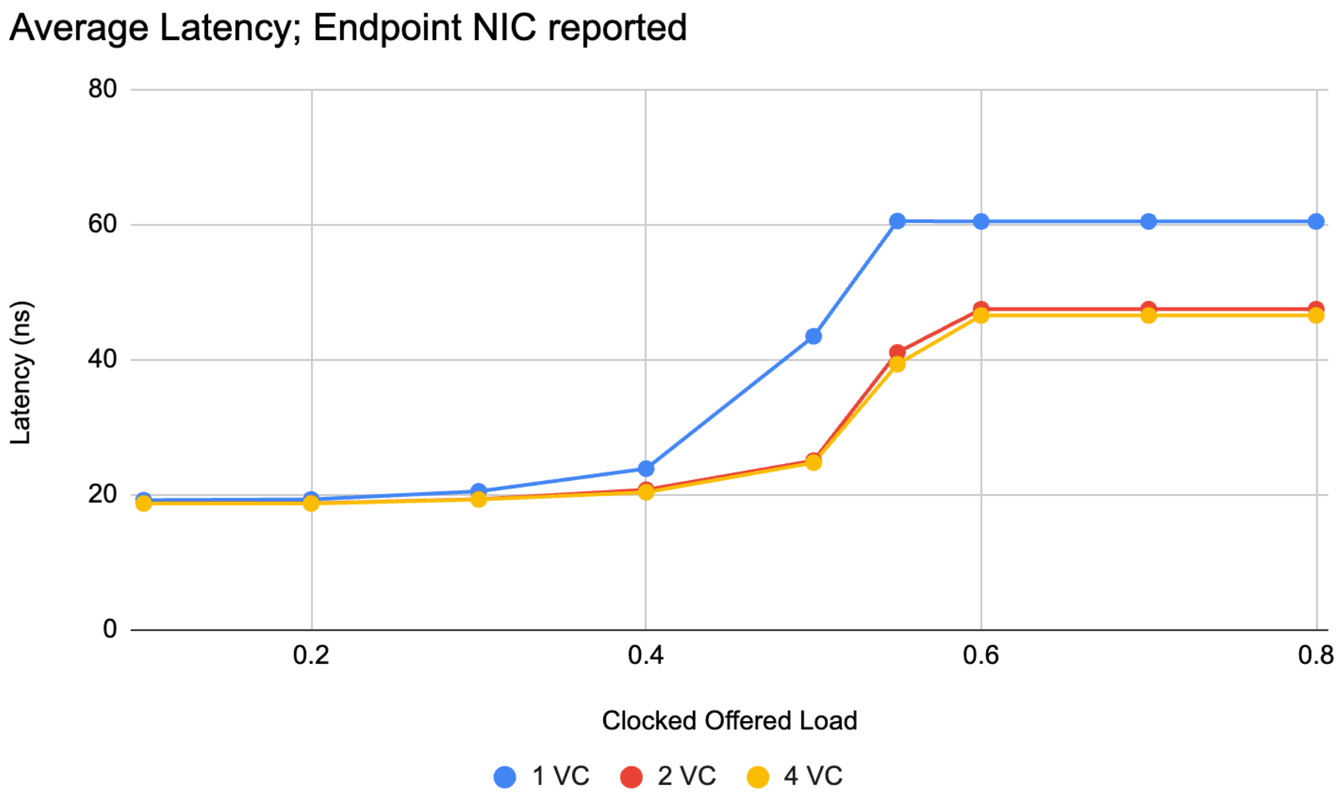
Jiang, N., Balfour, J., Becker, D. U., Towles, B., Dally, W. J., Michelogiannakis, G., & Kim, J. (2013). A detailed and flexible cycle-accurate Network-on-Chip simulator. In *Ispass 2013 IEEE International Symposium on Performance Analysis of Systems and Software* (pp. 86–96). https://doi.org/10.1109/ISPASS.2013.6557149

Num. Cycles Crossbar Blocked

Sum cycles crossbar blocked (all routers, all ports)			
Offered Load	Mordred COL	Merlin OL	Kingsley OL
0.1	0	121,844	176,401
0.2	0	242,858	351,445
0.3	117	372,092	537,154
0.4	9,905	544,056	782,848
0.5	173,671	831,516	1,157,086
0.6	294,466	1,360,116	1,878,379
0.7	294,466	2,160,511	3,010,965
0.8	294,466	3,355,002	5,428,789

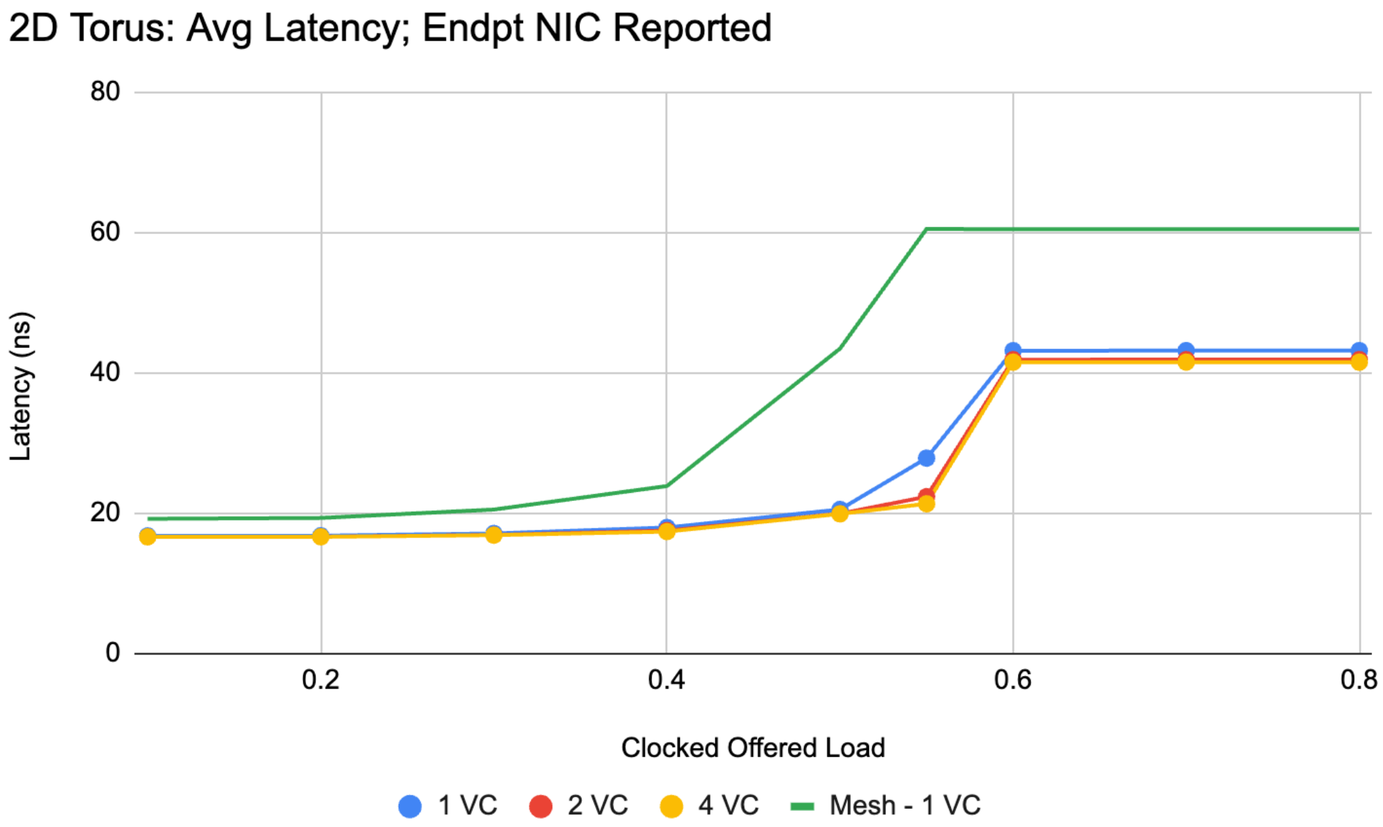
- Mordred significantly lower
- Highly likely to be a measurement difference; actively looking into

Avg. Latency – Multiple VCs



- Mordred: Test the use of multiple VCs
- Obvious improvement going from 1 to 2 VCs

New Topology: 2D Torus (3x3)



- Lower latency than Mesh; 1VC is notably lower than the Mesh
- Saturation curve pushed to a higher load

What's Next

- **Continue with our internal requirements**
  - Allow non-flit width channels
  - Enable priority based routing (cache coherency)
  - Configurable latency through the router
  - Test multiple VNs
  - More testing: different traffic patterns, packet size mix, etc.
  - Longer term: allow flits to bypass the router (OpenSMART like), flit based switching, performance profiling and improvements, etc.

- **MemHierarchy Integration**
- **Merlin Integration**
- **Full test suite**
- **Continue adding additional subcomponents (particularly allocators)**

Backup

sst-info Router Output

```
Components (3 total)
   Component 0: simple_rtr
      Description: Simple NoC router component
      ELI version: 0.9.0
      Compiled on: Oct 10 2025 15:05:56, using file: /Users/tdysart/git/mordred/src/SimpleRTR.h
      Category: NETWORK COMPONENT
      Parameters (9 total)
         verbose: Sets the output verbosity  [5]
         id: ID of the router  [<required>]
         clock: Clock frequency of the router  [1GHz]
         num_ports: Number of ports on the router  [3]
         num_local_ports: Number of local ports  [1]
         num_vcs: Number of virtual channels.  [1]
         flit_size: Flit size specified in either b or B (can include SI prefix).  [32b]
         input_buf_size: Size of per-VC input buffers specified in b or B (can include SI prefix).  [<required>]
         output_buf_size: Size of per-VC output buffer specified in b or B (can include SI prefix).  [<required>]
      Ports (1 total)
         port%(portnum)d: Port id.
      SubComponent Slots (4 total)
         topology: Topology and routing subcomponent [SST::Mordred::TopologyAPI]
         portcontrol: PortControl blocks; loaded anonymously [SST::Mordred::RtrPortControlAPI]
         vc_alloc: VC allocator [SST::Mordred::VcAllocAPI]
         arbiter: Arbitration scheme/model [SST::Mordred::ArbAPI]
      Statistics (3 total)
         xbar_idle: For each receiving port, num cycles with an idle crossbar,  (units = "unitless") Enable level = 3
         xbar_blocked: For each receiving port, num cycles crossbar blocked,  (units = "unitless") Enable level = 3
         flit_unavailable: Port does not have the flit to send thru the crossbar,  (units = "unitless") Enable level = 3
      Profile Points (0 total)
      Attributes (0 total)
```

Python Script - Mesh

```python
# There are local_ports endpoints connected to each router
# rtr_id is expected to go linearly from 0-((x*y)-1)
# rtr_id is also expected to be x-dominant (e.g, router id xDim has location x=0,y=1)
# links are expected to be ordered as n,e,s,w
def createMesh(x_size, y_size, local_ports):
    # Create the routers
    rtr_id = 0
    nports = 4 + local_ports
    rtr_params = {
        "num_ports": nports,
        "num_local_ports" : local_ports,
    }
    for y in range(y_size):
        for x in range(x_size):
            rtr = sst.Component("rtr_%d_%d"%(x, y), "mordred.simple_rtr")
            rtr.addParam("id", rtr_id)
            rtr.addParams(FixedRtrParams)
            rtr.addParams(rtr_params)
            rtr_id += 1
            rtr_topo = rtr.setSubComponent( "topology", "mordred.MeshTopology" )
            rtr_topo.addParams({
                "xDim" : x_size,
                "yDim" : y_size
            })
            # north links
            rtr_portnum = 0
            rtr_portname = "port" + str(rtr_portnum)
            if y != y_size - 1:
                rtr.addLink(getLink("rtr_%d_%d"%(x,y), "rtr_%d_%d"%(x,y+1)), rtr_portname, "800ps")

            # east links
            rtr_portnum += 1
            rtr_portname = "port" + str(rtr_portnum)
            if x != x_size - 1:
                rtr.addLink(getLink("rtr_%d_%d"%(x,y), "rtr_%d_%d"%(x+1,y)), rtr_portname, "800ps")

            # south links
            rtr_portnum += 1
            rtr_portname = "port" + str(rtr_portnum)
            if y != 0:
                rtr.addLink(getLink("rtr_%d_%d"%(x,y-1), "rtr_%d_%d"%(x,y)), rtr_portname, "800ps")

            # west links
            rtr_portnum += 1
            rtr_portname = "port" + str(rtr_portnum)
            if x != 0:
                rtr.addLink(getLink("rtr_%d_%d"%(x-1,y), "rtr_%d_%d"%(x,y)), rtr_portname, "800ps")
            rtr_portnum += 1
```

```python
            # local ports
            for k in range(local_ports):
                lcl_portname = "port" + str(k+rtr_portnum)
                # create endpoint
                ep_name = "local_ep_%d_%d_%d"%(x,y,k)
                ep_num = (x*y_size*local_ports) + (y*local_ports) + k
                num_eps = x_size * y_size * local_ports
                print("%s Created endpoint %d with num_eps %d"%(ep_name, ep_num, num_eps))
                if merlin_trafficgen == 0:
                    lcl_ep = sst.Component(ep_name, "merlin.offered_load")
                    lcl_ep.addParams(OfferedLoadParams)
                elif merlin_trafficgen == 1: # merlin_trafficgen == 1:
                    lcl_ep = sst.Component(ep_name, "merlin.background_traffic")
                else:
                    lcl_ep = sst.Component(ep_name, "merlin.clocked_offered_load")
                    lcl_ep.addParams(OfferedLoadParams)
                    lcl_ep.addParams(ClockedOfferedLoadParams)
                lcl_ep.addParam( "num_peers" , (num_eps) )
                lcl_ep.addParams(MatchingTrafficGenParams)
                lcl_ep_iface = lcl_ep.setSubComponent("networkIF", "mordred.mordredNIC")

                lcl_ep_iface.addParam("input_buf_size", "1kiB")
                lcl_ep_iface.addParam("output_buf_size", "1kiB")

                rtr.addLink(getLink("rtr_%d_%d"%(x, y), ep_name), lcl_portname, "800ps")
                lcl_ep_iface.addLink(getLink("rtr_%d_%d"%(x, y), ep_name), "port", "800ps")

                pattern_gen = lcl_ep.setSubComponent("pattern_gen", "merlin.targetgen.uniform")
                # Setting the following params seems to have no effect
                pattern_gen.addParams({
                    "min" : "0",
                    "max" : (num_eps-1),
                })
```

Python Script - FlattenedButterfly

```python
class FlattenedButterfly:
    def __init__(self, k, n):
        self.k = k
        self.n = n
        self.num_endpoints = pow(k, n)
        self.num_routers = floor(self.num_endpoints/k)
        self.radix = ( n * (k-1) ) + 1
        self.local_port_start = self.radix - self.k
        self.num_dims = n - 1
        self.flatfly_links = dict()
        self.routers = self.gen_routers()
        self.create_topo_links()
        self.endpoints = self.gen_endpoints()

    def gen_routers(self):

        routers = []
        for i in range(self.num_routers):
            rtr_name = "rtr_%d"%(i)
            routers.append(sst.Component(rtr_name, "mordred.simple_rtr"))
            routers[i].addParam( "id", i )
            routers[i].addParams(FixedRtrParams)
            routers[i].addParam( "num_ports", self.radix )
            routers[i].addParam( "num_local_ports", self.k )
            rtr_topo = routers[i].setSubComponent( "topology", "mordred.flattenedButterfly" )
            # TODO: Add topo params as needed
            rtr_topo.addParams({
                "k" : self.k,
                "n" : self.n,
            })
        return routers

# The range of d might be off a hair - paper has it as 1, self.n-1 but that blows up
# on a 4-ary, 2-fly.
    def create_topo_links(self):
        for d in range(self.num_dims):
            for m in range (self.k-1):
                for i in range(self.num_routers):
                    ff = floor(i/pow(self.k,d)) % self.k
                    j = i + ( ( m - ff ) * pow(self.k,d) )
                    if ( j != i ):
                        self.create_link(i,j)
```

```python
def create_link(self, i, j):
    if (i > j):
        i,j = j,i
    link_name, new_link = self.getFlatFlyLink("rtr_%d"%(i), "rtr_%d"%(j))
    if new_link:
        rtr_pname = "port" + str(getNextTopoPort("rtr_%d"%i))
        self.routers[i].addLink(link_name, rtr_pname, "800ps")
        rtr_pname = "port" + str(getNextTopoPort("rtr_%d"%j))
        self.routers[j].addLink(link_name, rtr_pname, "800ps")

def getFlatFlyLink(self, name1, name2):
    name = "link.%s_%s"%(name1, name2)
    if name not in self.flatfly_links:
        self.flatfly_links[name] = sst.Link(name)
        return self.flatfly_links[name], True
    return self.flatfly_links[name], False

def gen_endpoints(self):
    endpoints = []
    for i in range(self.num_routers):
        for j in range(self.k):
            portname = "port" + str(self.local_port_start + j)
            ep_name = "ep_%d_%d"%(i,j)
            ep_num = i*self.k + j
            ep = sst.Component(ep_name, "merlin.offered_load")
            ep.addParams({
                "num_peers" : self.num_endpoints, "link_bw" : "500MB/s",
                "linkcontrol" : "mordred.mordredNIC", "buffer_size" : "1kiB",
                "packet_size" : "16B", "pattern" : "merlin.targetgen.uniform",
                "offered_load" : "0.5", "warmup_time" : "1us",
                "collect_time" : "500us", "drain_time" : "50us"
            })
            endpoints.append(ep)
            ep_iface = ep.setSubComponent( "networkIF", "mordred.mordredNIC" )
            ep_iface.addParams({
                "input_buf_size" : "1kB", "output_buf_size" : "1kB",
            })
            rtr_id = "rtr_%d"%i
            ep_id = "ep_%d"%j
            link_name, new_link = self.createEpRtrLink(rtr_id, ep_id)
            if not new_link:
                print("WARN Failed to create ep link")
            self.routers[i].addLink(link_name, portname, "800ps")
            ep_iface.addLink(link_name, "port", "800ps")
            pattern_gen = ep.setSubComponent("pattern_gen", "merlin.targetgen.uniform")

def createEpRtrLink(self, rtr_id, ep_id):
    name = "link.%s_%s"%(rtr_id, ep_id)
    if name not in self.flatfly_links:
        self.flatfly_links[name] = sst.Link(name)
        return self.flatfly_links[name], True
    return self.flatfly_links[name], False
```